SynthWorks

VHDL Training Experts

TranscriptPkg Quick Ref

1. Overview

All OSVVM printing uses the transcript capability. If the TranscriptFile is open, all OSVVM printing will go to it. If the TranscriptFile is not open all OSVVM printing will go OUTPUT (stdout). If the TranscriptFile is open and transcript mirroring is enabled, printing will go to both the TranscriptFile and to OUTPUT.

The following shows the basic use model. Set the test name (using SetTestName). Open the transcript file (using TranscriptOpen). Mirror the transcript file to OUTPUT (SetTranscriptMirror). Print using the transcripting capability. Close the transcript file (TranscriptClose) just before the end of the test. Generate reports (using EndOfTestReports). Stop the test case (using std.env.stop).

```
library osvvm ;
context osvvm.OsvvmContext ;
. . .
architecture Test1 of TestCtrl is
begin
  ControlProc : process
  begin
    SetTestName("Tb_SendGet_1") ;
    TranscriptOpen ;           --
Print to Tb_SendGet_1.log
    SetTranscriptMirror ;      -- Also
print to OUTPUT
    PrintLine("Print 1") ;
    swrite(buf, "Write 1") ;
    writeline(buf) ;
    . . .
    TranscriptClose ;
    EndOfTestReports ;
    std.env.stop ;
  end process ControlProc ;
```

2. TranscriptOpen & TranscriptClose

TranscriptOpen opens the TranscriptFile. There are a number of variations (overloaded version) of TranscriptOpenWhen, however, the simplest is the following. It opens a file named TestCaseName.log, where TestCaseName is the name used in SetTestName.

```
procedure TranscriptOpen ;
```

TranscriptClose closes the TranscriptFile.

```
procedure TranscriptClose ;
```

When FILE_OPEN_KIND is not specified in the call, the first call to TranscriptOpen will open the file in WRITE_MODE. If the transcript is closed (with TranscriptClose) and then reopened, it will be opened with APPEND_MODE.

3. SetTranscriptMirror

SetTranscriptMirror sets mirror mode in which case all OSVVM printing goes to both the TranscriptFile and std.textio.OUTPUT.

```
procedure SetTranscriptMirror (A :
boolean := TRUE) ;
. . .
SetTranscriptMirror ;
```

4. WriteLine

WriteLine writes the buffer using the transcript capability.

```
swrite(buf, "A Message") ;
writeline(buf) ;
```

5. Print

Print writes a string using the transcript capability.

```
procedure Print(s : string) ;
. . .
Print("Another Message") ;
```

6. Blank

Blank puts a variable number of blank lines in the output.

```
procedure Blank(count : natural := 1);
. . .
BlankLine(5) ; -- prints 5 blank lines
```

7. Printing with Prefix

The following print operations prefix the printed lines with OSVVM_PRINT_PREFIX.

7.1 PrintLine

Print prints the string using the transcript capability. Like Log, print allows message filtering using Level and AlertLogID. Unlike Log, PrintLine does not print the time, "log", or the Log level.

```
procedure PrintLine (s : string) ;
procedure PrintLine (s : string ;
Level : LogType) ;
procedure PrintLine (AlertLogID :
AlertLogIDType ; s : string ; Level :
LogType);
```

7.2 PrintHeader

Print prints the string plus prefix and suffix lines to frame the message and make it stand out. Each character in the prefix and suffix indicates a line to print. If the character is a space, then print a blank line, otherwise, repeat that character OSVVM_LINE_LENGTH times. Like Log, PrintHeader supports message filtering using Level and AlertLogID.

```
procedure PrintHeader (s, Prefix,
Suffix : string) ;
procedure PrintHeader (s, Prefix,
Suffix : string ; Level : LogType ) ;
procedure PrintHeader (AlertLogID :
AlertLogIDType ; s, Prefix, Suffix :
string ; Level : LogType ) ;
```

There is also a short form that uses OSVVM_HEADER_PREFIX (default " ") and OSVVM_HEADER_SUFFIX (default "*") instead of having a Prefix and Suffix parameter.

```
procedure PrintHeader (s : string) ;
procedure PrintHeader (s : string ;
    Level : LogType ) ;
procedure PrintHeader (AlertLogID :
    AlertLogIDType; s : string;
    Level : LogType);
```

7.3 HeaderLine

HeaderLIne prints a string of characters that indicate a line to print. Each line printed is prefixed by OSVVM_PRINT_PREFIX. If the character is a space, then print a blank line, otherwise, repeat that character OSVVM_LINE_LENGTH times. In addition, like Log, HeaderLine supports message filtering using Level and AlertLogID.

```
procedure HeaderLine (S : string) ;
procedure HeaderLine (S : string;
  Level : LogType;
  Enable : boolean := FALSE ) ;
procedure HeaderLine (AlertLogID :
  AlertLogIDType; S : string;
  Level : LogType := ALWAYS;
  Enable : boolean := FALSE) ;
```

7.4 BlankLine

BlankLine puts a variable number of blank lines in the output. Each line is prefixed by OSVVM_PRINT_PREFIX. Blank lines can also be filtered with Log levels and AlertLogID.

```
procedure BlankLine (
  count : natural := 1) ;
procedure BlankLine (
  count : natural; Level : LogType;
  Enable : boolean := FALSE);
procedure BlankLine (AlertLogID :
  AlertLogIDType ; count : natural;
  Level : LogType := ALWAYS;
  Enable : boolean := FALSE);
```

8. IsTranscriptOpen

IsTranscriptOpen returns TRUE when the TranscriptFile is open.

```
if IsTranscriptOpen then
  swrite(buf, "Yet Another Message") ;
  writeline(TranscriptFile, buf) ;
end if;
```

9. IsTranscriptMirrored

IsTranscriptMirrored returns true when the transcript is mirrored.

```
if IsTranscriptMirrored then
  . . .
```


SynthWorks Design Inc.
VHDL Intro, RTL, and Verification Training
11898 SW 128th Ave. Tigard OR 97223 (800)-505-8435
http://www.SynthWorks.com jim@synthworks.com